



合肥师范学院
HEFEI NORMAL UNIVERSITY

实验报告

课程名称: web 程序设计

学院: 计算机与人工智能学院

专业: 计算机科学与技术

学号: 2410211124

姓名: 朱品赞

指导教师: 张由明

2025 年 12 月 26 日

实验二 Servlet 基础技术实验

一、实验目的

1. 掌握 Servlet 的基本原理和生命周期
2. 学会使用 @WebServlet 注解配置 Servlet
3. 理解 HttpServletRequest 和 HttpServletResponse 的使用
4. 掌握请求参数获取和响应重定向
5. 理解 HttpSession 会话管理和用户状态保持

二、实验内容及步骤

2.1 接收用户注册请求的业务处理

目标

实现用户注册功能，包括参数验证、用户名唯一性检查和数据持久化。

步骤

Step 1: 创建 SysUserController 类

```
package com.zpz.schedule.controller;

import com.zpz.schedule.pojo.SysUser;
import com.zpz.schedule.service.SysUserService;
import com.zpz.schedule.service.impl.SysUserServiceImpl;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/user/*")
public class SysUserController extends BaseController {
    private SysUserService sysUserService = new SysUserServiceImpl();

    // 注册、登录、登出方法实现
}
```

Step 2: 编写注册方法

```

public void register(HttpServletRequest req, HttpServletResponse resp)
throws IOException {
    // 1. 获取请求参数 (用户名、密码、密码确认)
    String username = req.getParameter("username");
    String password = req.getParameter("password");
    String repassword = req.getParameter("repassword");

    // 2. 参数验证
    if (username == null || password == null ||
!password.equals(repassword)) {
        // 参数不合法, 重定向到错误页面
        resp.sendRedirect(req.getContextPath() + "/registError.html");
        return;
    }

    // 3. 创建用户对象
    SysUser sysUser = new SysUser();
    sysUser.setUsername(username);
    sysUser.setUserPwd(password);

    // 4. 调用业务层处理用户注册
    boolean result = sysUserService.register(sysUser);

    // 5. 根据结果返回相应页面
    if (result) {
        // 注册成功, 重定向到成功页面
        resp.sendRedirect(req.getContextPath() + "/registSuccess.html");
    } else {
        // 用户名已存在, 重定向到提示页面
        resp.sendRedirect(req.getContextPath() +
"/registUsernameExist.html");
    }
}

```

关键技术点：

- req.getParameter(String name) - 获取 POST 请求中的参数
- resp.sendRedirect(String url) - 重定向响应，浏览器将自动跳转到新页面
- req.getContextPath() - 获取应用上下文路径，确保路径正确

2.2 接收用户登录请求，完成的登录业务接口

目标

实现用户登录功能，包括身份验证、会话管理和用户信息存储。

步骤

Step 1: 编写登录方法

```
public void login(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    // 1. 获取请求参数 (用户名、密码)
    String username = req.getParameter("username");
    String password = req.getParameter("password");

    // 2. 调用业务层验证用户身份
    SysUser sysUser = sysUserService.login(username, password);

    // 3. 根据验证结果处理响应
    if (sysUser != null) {
        // 认证成功
        // 3.1 获取当前会话对象
        HttpSession session = req.getSession();

        // 3.2 在会话中存储已登录用户信息
        session.setAttribute("loginUser", sysUser);

        // 3.3 重定向到日程管理页面
        resp.sendRedirect(req.getContextPath() + "/showSchedule.html");
    } else {
        // 认证失败 (用户不存在或密码错误)
        resp.sendRedirect(req.getContextPath() + "/loginError.html");
    }
}
```

Step 2: 编写登出方法

```
public void logout(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    // 1. 获取当前会话
    HttpSession session = req.getSession();

    // 2. 销毁会话, 清除所有会话属性
    session.invalidate();

    // 3. 重定向到登录页面
    resp.sendRedirect(req.getContextPath() + "/login.html");
}
```

关键技术点:

- req.getSession() - 获取或创建 HttpSession 对象
- session.setAttribute(String name, Object value) - 在会话中存储属性
- HttpSession 会自动生成 JSESSIONID, 通过 Cookie 在客户端和服务端间传递
- session.invalidate() - 完全销毁会话, 清除所有数据

2.3 业务层实现

SysUserServiceImpl 类:

```
public class SysUserServiceImpl implements SysUserService {
    private SysUserDao sysUserDao = new SysUserDaoImpl();

    @Override
    public boolean register(SysUser sysUser) {
        // 1. 检查用户名是否已存在
        SysUser existUser =
sysUserDao.findByUsername(sysUser.getUsername());
        if (existUser != null) {
            return false; // 用户名已存在
        }

        // 2. MD5加密密码
        sysUser.setUserPwd(MD5Util.encrypt(sysUser.getUserPwd()));

        // 3. 保存用户到数据库
        return sysUserDao.addSysUser(sysUser) > 0;
    }

    @Override
    public SysUser login(String username, String password) {
        // 1. 按用户名查询用户
        SysUser sysUser = sysUserDao.findByUsername(username);
        if (sysUser == null) {
            return null; // 用户不存在
        }

        // 2. 对输入密码进行MD5加密
        String encryptPwd = MD5Util.encrypt(password);

        // 3. 比对密码
        if (sysUser.getUserPwd().equals(encryptPwd)) {
            return sysUser; // 密码正确
        }
        return null; // 密码错误
    }
}
```

```
}  
}
```

三、实验结果

3.1 用户注册流程

流程说明：

```
用户访问 /register.html  
↓  
填写注册信息（用户名、密码、确认密码）  
↓  
点击"注册"按钮，发送 POST 请求到 /user/register  
↓  
SysUserController.register() 处理请求  
↓  
参数验证 → 检查用户名唯一性 → MD5加密密码 → 保存数据库  
↓  
注册成功 → 重定向到 /registSuccess.html  
↓  
注册失败 → 重定向到 /registUsernameExist.html
```

数据库变化：

```
-- 执行 INSERT 语句  
INSERT INTO sys_user (username, user_pwd) VALUES ('zhangsan',  
'5f4dcc3b5aa765d61d8327deb882cf99')  
  
-- 查询结果  
SELECT uid, username, user_pwd FROM sys_user WHERE username = 'zhangsan';  
  
-- 输出  
uid | username | user_pwd  
----|-----|-----  
1   | zhangsan | 5f4dcc3b5aa765d61d8327deb882cf99
```

3.2 用户登录流程

流程说明：

```
用户访问 /login.html  
↓  
填写登录信息（用户名、密码）
```

↓
点击"登录"按钮, 发送 POST 请求到 /user/login
↓
SysUserController.login() 处理请求
↓
查询用户 → MD5加密输入密码 → 比对密码
↓
认证成功 → 创建 Session → 重定向到 /showSchedule.html
↓
认证失败 → 重定向到 /loginError.html

Session 中的数据:

```
// 成功登录后, Session 中存储的内容
HttpSession session = req.getSession();
// session.getAttribute("loginUser") 返回:
SysUser {
    uid = 1,
    username = "zhangsan",
    userPwd = "5f4dcc3b5aa765d61d8327deb882cf99"
}
```

3.3 受保护资源访问流程

日程管理页面访问流程:

用户访问 /showSchedule.html
↓
LoginFilter.doFilter() 拦截请求
↓
检查 Session 中是否存在 loginUser
↓
已登录 → 继续执行, 显示日程列表页面
↓
未登录 → 重定向到 /login.html (需要先登录)

四、常见问题

问题 1: 为什么要设置请求和响应的字符编码?

现象: 中文参数显示为乱码

原因:

- 浏览器发送的请求默认使用页面编码
- 服务器需要明确指定编码方式来解析参数
- 响应也需要指定编码，确保浏览器正确显示中文

解决方案：

```
req.setCharacterEncoding("UTF-8");           // 设置请求字符编码  
resp.setContentType("text/html;charset=UTF-8"); // 设置响应字符编码
```